Day 31 – Regular Expressions (RegEx)

What is Regular Expression (RegEx)?

A **Regular Expression (RegEx)** is a sequence of characters that defines a search pattern. It is used to search, match, extract, validate, and replace text efficiently.

Python provides the built-in `re` module to work with Regular Expressions.

Regular Expressions are widely used in web development, data validation, data cleaning, log analysis, and text processing.

Importing the `re` Module

Before using Regular Expressions, import the `re` module.

```python
import re
```

Why Use Regular Expressions?

Regular Expressions are mainly used for:

- Email Validation
- Mobile Number Validation
- Password Validation
- Username Validation
- Aadhaar Validation
- PAN Card Validation
- URL Validation
- PIN Code Validation
- Search and Replace
- Data Extraction
- Text Cleaning
- Log File Analysis
- Parsing Text Files

Basic Example

```python
import re
text = "Order ID: 12345"
result = re.search(r"\d+", text)
print(result.group())
```

Output

```
None
12345
```

Why Do We Use `r""` in Regular Expressions?

The prefix `r` before a string represents a **Raw String**.

A raw string tells Python to treat backslashes (`\`) as normal characters instead of escape characters.

Without a raw string:

```python
"\n"
```

Python treats `\n` as a newline.

With a raw string:

```python
r"\n"
```

Python treats `\n` as two separate characters (`\` and `n`).

Example

```python
import re
text = "Age is 25"
result = re.search(r"\d+", text)
print(result.group())
```

Here,

- `\d` represents a digit.
- `+` means one or more digits.

Using `r""` makes Regular Expressions easier to write and understand.

Regular Expression Functions

Function	Description	Syntax
`re.match()`	Checks the pattern only at the beginning of the string	`re.match(pattern, string)`
`re.search()`	Searches the entire string and returns the first match	`re.search(pattern, string)`
`re.findall()`	Returns all matches as a list	`re.findall(pattern, string)`
`re.finditer()`	Returns an iterator containing match objects	`re.finditer(pattern, string)`
`re.fullmatch()`	Matches the entire string	`re.fullmatch(pattern, string)`
`re.sub()`	Replaces matched text	`re.sub(pattern, replacement, string)`
`re.split()`	Splits a string based on a pattern	`re.split(pattern, string)`
`re.compile()`	Compiles a regular expression for repeated use	`re.compile(pattern)`
`re.escape()`	Escapes all special regex characters	`re.escape(string)`

1. re.match()

Purpose

`re.match()` checks whether the pattern matches **only at the beginning of the string**.

If the beginning matches, it returns a Match object.

Otherwise, it returns `None`.

Syntax

```python
re.match(pattern, string)
```

Usage

Use `re.match()` when the pattern must appear at the **start** of the string.

Example 1

```python
import re

result = re.match(r"Hello", "Hello World")

if result:
    print("Match Found")
else:
    print("No Match")
```

Output

```
Match Found
```

Example 2

```python
import re

result = re.match(r"Hello", "Hi Hello")

if result:
    print("Match Found")
else:
    print("No Match")
```

Output

```
No Match
```

2. re.search()

Purpose

`re.search()` searches the **entire string** and returns the **first occurrence** of the pattern.

If no match is found, it returns `None`.

Syntax

```python
re.search(pattern, string)
```

Usage

Use `re.search()` when the pattern can appear **anywhere** inside the string.

Example

```python
import re
```

```
text = "Age is 25"
result = re.search(r"\d+", text)
print(result.group())
```

Output

```
None
25
```

Example

```
Python
import re
text = "Python 3.13"
result = re.search(r"\d+", text)
print(result.group())
```

Output

```
None
3
```

3. re.findall()

Purpose

`re.findall()` returns **all matching patterns** as a list.

If no match is found, it returns an empty list.

Syntax

```
Python
re.findall(pattern, string)
```

Usage

Use `re.findall()` when you need every occurrence of a pattern.

Example

```python
import re
numbers = re.findall(r"\d+", "10 20 30 40")
print(numbers)
```

Output

```
None
['10', '20', '30', '40']
```

Example

```python
import re
words = re.findall(r"[A-Za-z]+", "Python 3 Java 17 C++")
print(words)
```

Output

```
None
['Python', 'Java', 'C']
```

4. re.finditer()

Purpose

`re.finditer()` returns an **iterator of Match objects**.

Each Match object contains additional information such as:

- Matched text
- Starting index
- Ending index

Syntax

```python
re.finditer(pattern, string)
```

Usage

Use `re.finditer()` when you need both the matched value and its position.

Example

```python
import re
text = "10 20 30"
result = re.finditer(r"\d+", text)
for i in result:
    print(i.group(), i.start())
```

Output

```
10 0
20 3
30 6
```

Example

```python
import re
text = "Python Java C"
result = re.finditer(r"[A-Za-z]+", text)
for i in result:
    print(i.group(), i.start(), i.end())
```

Output

```
Python 0 6
```

```
Java 7 11
C 12 13
```

5. re.fullmatch()

Purpose

`re.fullmatch()` checks whether the **entire string** matches the given pattern.

If even one character does not satisfy the pattern, it returns `None`.

Syntax

Python
```
re.fullmatch(pattern, string)
```

Usage

Use `re.fullmatch()` for data validation such as:

- Email Validation
- Mobile Number Validation
- Password Validation
- PAN Validation
- Aadhaar Validation

Example

Python
```
import re
result = re.fullmatch(r"\d{10}", "9876543210")
if result:
    print("Valid")
else:
    print("Invalid")
```

Output

```
None
Valid
```

Example

```python
import re
result = re.fullmatch(r"\d{10}", "98765")
if result:
    print("Valid")
else:
    print("Invalid")
```

Output

```
None
Invalid
```

6. re.sub()

Purpose

`re.sub()` replaces all occurrences of a pattern with the specified replacement string.

Syntax

```python
re.sub(pattern, replacement, string)
```

Usage

Use `re.sub()` when you want to replace matched text.

Example 1

```python
import re
text = "I love cat"
result = re.sub(r"cat", "dog", text)
print(result)
```

Output

```
None
I love dog
```

Example 2

```python
import re
text = "Python123"
result = re.sub(r"\d", "*", text)
print(result)
```

Output

```
None
Python***
```

7. re.split()

Purpose

`re.split()` splits a string wherever the given pattern matches.

Syntax

```python
re.split(pattern, string)
```

Usage

Use `re.split()` when the separator is not a fixed character.

Example 1

```python
import re
text = "apple,banana;orange"
result = re.split(r"[,;]", text)
print(result)
```

Output

```
None
['apple', 'banana', 'orange']
```

Example 2

```python
import re
text = "Python Java,C++;SQL"
result = re.split(r"[ ,;]+", text)
print(result)
```

Output

```
None
['Python', 'Java', 'C++', 'SQL']
```

8. re.compile()

Purpose

`re.compile()` compiles a regular expression into a pattern object so that it can be reused multiple times.

Compiling once improves readability and performance when the same pattern is used repeatedly.

Syntax

```python
re.compile(pattern)
```

Usage

Use `re.compile()` when the same pattern is used many times in a program.

Example 1

```python
import re
pattern = re.compile(r"\d+")
result = pattern.findall("10 20 30 40")
print(result)
```

Output

```
['10', '20', '30', '40']
```

Example 2

```python
import re
pattern = re.compile(r"[A-Za-z]+")
text = "Python 3 Java 17"
for i in pattern.finditer(text):
    print(i.group())
```

Output

```
Python
Java
```

9. re.escape()

Purpose

`re.escape()` adds a backslash (`\`) before every special Regular Expression character.

This treats special characters as normal characters.

Syntax

```python
re.escape(string)
```

Usage

Use `re.escape()` when searching for strings that contain regex special characters such as `.`, `*`, `+`, `?`, `(`, `)` and others.

Example 1

```python
import re
text = "price(100)"
print(re.escape(text))
```

Output

```
price\(100\)
```

Example 2

```python
import re
text = "file.txt"
print(re.escape(text))
```

Output

```
None
file\.txt
```

Difference Between re.match(), re.search() and re.fullmatch()

Function	Checks	Example	Result
`re.match()`	Beginning of the string only	`"Hello World"`	Match
`re.search()`	Entire string	`"Hi Hello"`	Match
`re.fullmatch()`	Entire string must match	`"Hello"`	Match

Example

```python
import re
text = "Hello World"
print(re.match(r"Hello", text))
print(re.search(r"World", text))
print(re.fullmatch(r"Hello World", text))
```

Output

```
None
<re.Match object>
<re.Match object>
<re.Match object>
```

Example

```python
import re
text = "Hello World"
print(re.fullmatch(r"Hello", text))
```

Output

```
None
```

None

Difference Between re.findall() and re.finditer()

re.findall()	re.finditer()
Returns a list	Returns an iterator
Returns only matched values	Returns Match objects
Cannot get positions	Can get start() and end() positions
Simpler to use	More powerful

Example using `findall()`

```python
import re
text = "10 20 30"
print(re.findall(r"\d+", text))
```

Output

```
None
['10', '20', '30']
```

Example using `finditer()`

```python
import re
text = "10 20 30"
for i in re.finditer(r"\d+", text):
    print(i.group(), i.start())
```

Output

```
None
10 0
20 3
30 6
```

RegEx Metacharacters

Metacharacters are special symbols that have a special meaning in Regular Expressions.

Basic Metacharacters

Symbol	Meaning	Example
`.`	Matches any single character except newline	`h.t`
`^`	Beginning of the string	`^Hello`
`$`	End of the string	`World$`
`*`	Zero or more occurrences	`ab*`
`+`	One or more occurrences	`ab+`
`?`	Zero or one occurrence	`colou?r`
`` ` ``	`` ` ``	OR operator

Examples

`.` (Any Character)

```python
import re
print(re.findall(r"h.t", "hat hit hot"))
```

Output

```
None
['hat', 'hit', 'hot']
```

^ (Beginning of String)

```python
import re
print(re.search(r"^Hello", "Hello World"))
```

Output

```
None
<re.Match object>
```

$ (End of String)

```python
import re
print(re.search(r"World$", "Hello World"))
```

Output

```
None
<re.Match object>
```

* (Zero or More)

```python
import re
print(re.findall(r"ab*", "a abb abbb"))
```

Output

```
None
['a', 'abb', 'abbb']
```

+ (One or More)

```python
import re
print(re.findall(r"ab+", "a abb abbb"))
```

Output

```
None
['abb', 'abbb']
```

? (Zero or One)

```python
import re
print(re.findall(r"colou?r", "color colour"))
```

Output

```
None
['color', 'colour']
```

| (OR Operator)

```python
import re
print(re.findall(r"cat|dog", "cat dog bird"))
```

Output

```
None
['cat', 'dog']
```

Character Classes

Character classes allow us to match one character from a specified set of characters.

Pattern	Meaning	Example
`[abc]`	Matches either `a`, `b`, or `c`	`[abc]`
`[^abc]`	Matches any character except `a`, `b`, or `c`	`[^abc]`
`[a-z]`	Matches any lowercase letter	`[a-z]`

`[A-Z]`	Matches any uppercase letter	`[A-Z]`
`[0-9]`	Matches any digit	`[0-9]`
`[A-Za-z]`	Matches any alphabet	`[A-Za-z]`
`[A-Za-z0-9]`	Matches any alphanumeric character	`[A-Za-z0-9]`

Example 1

```python
import re
print(re.findall(r"[abc]", "apple ball cat"))
```

Output

```
None
['a', 'b', 'a', 'c', 'a']
```

Example 2

```python
import re
print(re.findall(r"[a-z]", "Python123"))
```

Output

```
None
['y', 't', 'h', 'o', 'n']
```

Example 3

```python
import re
print(re.findall(r"[A-Z]", "Python JAVA"))
```

Output

```
None
['P', 'J', 'A', 'V', 'A']
```

Example 4

```
Python
import re
print(re.findall(r"[0-9]", "Python123"))
```

Output

```
None
['1', '2', '3']
```

Example 5

```
Python
import re
print(re.findall(r"[A-Za-z]", "Python123"))
```

Output

```
None
['P', 'y', 't', 'h', 'o', 'n']
```

Example 6

```
Python
import re
print(re.findall(r"[A-Za-z0-9]", "@Python123#"))
```

Output

```
None
['P', 'y', 't', 'h', 'o', 'n', '1', '2', '3']
```

Special Sequences

Special sequences are predefined character classes that make Regular Expressions shorter and easier to write.

Pattern	Meaning	Equivalent
`\d`	Any digit	`[0-9]`
`\D`	Any non-digit	`[^0-9]`
`\w`	Word character (letters, digits, underscore)	`[A-Za-z0-9_]`
`\W`	Non-word character	`[^A-Za-z0-9_]`
`\s`	Whitespace character	Space, Tab, Newline
`\S`	Non-whitespace character	Any visible character
`\b`	Word boundary	Beginning or end of a word
`\B`	Not a word boundary	Inside a word

Example 1 – `\d`

Python

```python
import re
print(re.findall(r"\d", "Python123"))
```

Output

None

```
['1', '2', '3']
```

Example 2 – `\D`

Python

```python
import re
print(re.findall(r"\D", "Python123"))
```

Output

```
None
['P', 'y', 't', 'h', 'o', 'n']
```

Example 3 – \w

```
Python
import re
print(re.findall(r"\w", "Python_123"))
```

Output

```
None
['P', 'y', 't', 'h', 'o', 'n', '_', '1', '2', '3']
```

Example 4 – \W

```
Python
import re
print(re.findall(r"\W", "Python@123#"))
```

Output

```
None
['@', '#']
```

Example 5 – \s

```
Python
import re
print(re.findall(r"\s", "Python Java SQL"))
```

Output

```
None
[' ', ' ']
```

Example 6 – \S

```python
import re
print(re.findall(r"\S", "Hi All"))
```

Output

```
None
['H', 'i', 'A', 'l', 'l']
```

Example 7 – \b

```python
import re
print(re.findall(r"\bcat\b", "cat category wildcat cat"))
```

Output

```
None
['cat', 'cat']
```

Example 8 – \B

```python
import re
print(re.findall(r"\Bon\B", "only on one"))
```

Output

```
None
['on']
```

Quantifiers

Quantifiers specify how many times a character or group should occur.

Pattern	Meaning	Example
`a{2}`	Exactly 2 times	`aa`
`a{3}`	Exactly 3 times	`aaa`
`a{2,5}`	2 to 5 times	`aa`, `aaa`, `aaaa`
`a{2,}`	2 or more times	`aa`, `aaa`, `aaaa`
`a*`	0 or more times	`a`, `aa`, `aaa`
`a+`	1 or more times	`a`, `aa`, `aaa`
`a?`	0 or 1 time	`a`

Example 1

Python

```python
import re
print(re.findall(r"a{2}", "a aa aaa aaaa"))
```

Output

None

```
['aa', 'aa', 'aa', 'aa']
```

Example 2

Python

```python
import re
print(re.findall(r"a{2,5}", "a aa aaa aaaa aaaaa"))
```

Output

None

```
['aa', 'aaa', 'aaaa', 'aaaaa']
```

Example 3

```python
import re
print(re.findall(r"ab*", "a ab abb abbb"))
```

Output

```
None
['a', 'ab', 'abb', 'abbb']
```

Example 4

```python
import re
print(re.findall(r"ab+", "a ab abb abbb"))
```

Output

```
None
['ab', 'abb', 'abbb']
```

Example 5

```python
import re
print(re.findall(r"colou?r", "color colour"))
```

Output

```
None
['color', 'colour']
```

Grouping

Grouping is used to treat multiple characters as a single unit.

Pattern	Meaning
`(abc)`	Creates a group
`(abc)+`	Repeats the group one or more times
`a	b`

Example 1

```python
import re
print(re.findall(r"(cat|dog)", "cat dog bird cat"))
```

Output

```
None
['cat', 'dog', 'cat']
```

Example 2

```python
import re
print(re.findall(r"(ab)+", "ab abab ababab"))
```

Output

```
None
['ab', 'ab', 'ab']
```

Example 3

```python
import re
print(re.findall(r"red|green|blue", "red yellow blue green"))
```

Output

```
None
['red', 'blue', 'green']
```

Form Validation Using Regular Expressions

Regular Expressions are widely used to validate user input in forms.

Common applications include:

- Email Validation
- Mobile Number Validation
- Password Validation
- Username Validation
- PAN Card Validation
- Aadhaar Validation
- PIN Code Validation
- URL Validation

1. Email Validation

Pattern

```
None
^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$
```

Explanation

- `^` → Beginning of the string
- `[A-Za-z0-9._%+-]+` → Username
- `@` → Mandatory symbol
- `[A-Za-z0-9.-]+` → Domain name
- `\.` → Dot
- `[A-Za-z]{2,}` → Domain extension (com, in, org, etc.)
- `$` → End of the string

Program

```python
import re
email = input("Enter Email: ")
if
re.fullmatch(r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$",
email):
    print("Valid Email")
else:
    print("Invalid Email")
```

Valid Examples

```
john@gmail.com
admin123@yahoo.in
python.user@gmail.org
```

Invalid Examples

```
john@gmail
@gmail.com
john.com
```

2. Mobile Number Validation

Pattern

```
^[6-9]\d{9}$
```

Explanation

- Must start with 6, 7, 8 or 9
- Must contain exactly 10 digits

Program

```python
import re
mobile = input("Enter Mobile Number: ")
if re.fullmatch(r"^[6-9]\d{9}$", mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

Valid Examples

```
9876543210
8123456789
7654321987
```

Invalid Examples

```
5876543210
98765
98765432101
```

3. Password Validation

Conditions

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character

Pattern

```
^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%^&+=!]).{8,}$
```

Program

```python
import re
password = input("Enter Password: ")
if
re.fullmatch(r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%^&+=!]).{
8,}$", password):
    print("Strong Password")
else:
    print("Weak Password")
```

Valid Examples

```
Python@123
Admin#456
Hello@2025
```

Invalid Examples

```
python123
ADMIN123
Python123
```

4. Username Validation

Conditions

- Only alphabets
- Digits
- Underscore (_)
- No spaces
- No special characters

Pattern

```
^[A-Za-z0-9_]+$
```

Program

```python
import re
username = input("Enter Username: ")
if re.fullmatch(r"^[A-Za-z0-9_]+$", username):
    print("Valid Username")
else:
    print("Invalid Username")
```

Valid Examples

```
teja_123
admin01
python_user
```

Invalid Examples

```
admin@123
user-name
my user
```

5. PAN Card Validation

Pattern

```
^[A-Z]{5}[0-9]{4}[A-Z]$
```

Explanation

- First 5 uppercase letters
- Next 4 digits
- Last 1 uppercase letter

Program

```python
import re
pan = input("Enter PAN Number: ")
if re.fullmatch(r"^[A-Z]{5}[0-9]{4}[A-Z]$", pan):
    print("Valid PAN Number")
else:
    print("Invalid PAN Number")
```

Valid Examples

```
ABCDE1234F
PQRSX6789K
```

Invalid Examples

```
abcde1234f
ABCDE12345
ABC1234DE
```

6. Aadhaar Validation

Pattern

```
^\d{12}$
```

Explanation

- Must contain exactly 12 digits

Program

```python
import re
aadhaar = input("Enter Aadhaar Number: ")
if re.fullmatch(r"^\d{12}$", aadhaar):
```

```
    print("Valid Aadhaar Number")
else:
    print("Invalid Aadhaar Number")
```

Valid Examples

```
None
123456789012
987654321098
```

Invalid Examples

```
None
12345
12345678901
1234567890123
```

7. PIN Code Validation

Pattern

```
None
^\d{6}$
```

Explanation

- Must contain exactly 6 digits

Program

```
Python
import re
pincode = input("Enter PIN Code: ")
if re.fullmatch(r"^\d{6}$", pincode):
    print("Valid PIN Code")
else:
    print("Invalid PIN Code")
```

Valid Examples

```
None
500081
560001
110001
```

Invalid Examples

```
None
50008
5000812
ABCDE1
```

8. URL Validation

Pattern

```
None
^(https?:\/\/)?(www\.)?[A-Za-z0-9.-]+\.[A-Za-z]{2,}$
```

Explanation

- `http://` or `https://` is optional
- `www.` is optional
- Domain name is mandatory
- Extension must contain at least two letters

Program

```python
import re
url = input("Enter URL: ")
if
re.fullmatch(r"^(https?:\/\/)?(www\.)?[A-Za-z0-9.-]+\.[A-Za-z]{2,
}$", url):
    print("Valid URL")
else:
    print("Invalid URL")
```

Valid Examples

None

```
https://codegnan.com
http://google.com
www.python.org
openai.com
```

Invalid Examples

None

```
google
https://
www.
abc
```